BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS

CSE3433 - SOFTWARE ARCHITECTURE

SEMESTER II: SESSION 2025/2026

Project Title:

MICROSERVICES-BASED DISASTER RELIEF COORDINATION PLATFORM

GROUP 16

COURSE LECTURER:

DR. MOHAMAD NOR HASSAN

NAME	MATRIC NUMBER
ADLY AZAMIN BIN AZMAN	S76094
MUHAMMAD UQAIL RAFIQIE BIN MOHD FARIS	S76227
AFIQ FAHIM BIN MOHD RIDZUAN	S76096

Table of Contents

Selected Architecture Style

A microservice design was chosen because it makes the software flexible and strong. Instead of building one giant application where everything is tangled together, the system is broken down into small independent pieces. Each piece focuses on doing just one specific job. This means that a single part can be changed, tested and launched on its own without breaking the rest of the software. If one piece stops working, the other parts keep running smoothly so users are not interrupted.

This setup also changes how information is stored. Instead of forcing the whole system to use one massive database that can easily get slow, each small piece gets its own separate database. This lets developers pick the best tools for each specific job because these pieces run separately in the cloud making it very easy to handle heavy traffic. If lots of people suddenly use one specific feature, more computer power can be added just to that one part. This keeps the software running fast and saves money because there is no need to pay for an upgrade to the entire system at once.

Problem Description

During emergencies like floods, earthquakes or disease outbreaks, it is very important for different groups to work well together. However, this is often hard to do. Many current disaster systems use older single block designs. These older designs are not flexible enough to handle the sudden rush of users and information that happens during a crisis.

A big problem with these older systems is that they can slow down or crash completely when too many people use them at once. If many victims try to report problems or ask for help at the exact same time, the system can freeze. This can cause dangerous delays for rescue teams trying to save lives.

Another issue is poor communication between groups like the government, charities, and volunteers. When information is not shared instantly, rescue workers might end up doing the same job twice or they might completely miss areas that badly need help.

Also, updating these traditional systems is very hard because all the parts are tightly linked together. Changing just one small piece can break the whole system. This makes fixing or improving the software slow and risky during an emergency.

Because of these problems, a better, more flexible system is needed. A modern design must be able to handle sudden spikes in users, share information instantly and allow different parts of the system to work and update on their own without breaking the rest of the platform.

System Objectives

The main goal of this project is to build a disaster relief platform using a microservice design. This will be done in three clear steps which are studying the problem, planning the design, and building the software.

The first step is to study the problems with current disaster systems. This means looking at why older systems crash, slow down or fail to share information during emergencies. By understanding these issues, it is easier to figure out exactly what the new system needs to do to solve real-world problems.

The next step is to plan the new system using small, independent parts. The software will be divided into separate services, such as one for reporting emergencies, another for tracking supplies and one for organizing volunteers. This plan ensures the system can easily grow and stay online even when many people use it at once during a crisis.

The final step is to build the software based on this plan. This involves creating the small services, connecting them so they can talk to each other and making sure everything works smoothly. The goal is to create a reliable platform that is easy to use and helps rescue teams do their jobs better.

Together, these steps will lead to a system that is carefully researched, well planned and successfully built to make disaster relief much more effective.

Initial Functional Requirements

1. User Access & Security

FR1: The system shall allow victims, volunteers, and agencies to create and manage their own accounts.

FR2: The system shall use a single entry point (API Gateway) to keep all services secure.

2. Emergency Reporting

FR3: The system shall allow users to send real-time reports with their location and the type of emergency.

FR4: The system shall allow the Reporting Service to scale independently to handle traffic spikes

3. Logistics & Supplies

FR5: The system shall provide a specific service to track and manage the distribution of food and medicine.

FR6: The system shall use a separate database for supply tracking to keep the data organized and fast.

4. Volunteer Coordination

FR7: The system shall allow volunteers to sign up and be assigned to specific rescue tasks.

FR8: The system shall allow the volunteer service to be updated without stopping the rest of the platform.

5. Communication

FR9: The system shall share information instantly between the government, charities, and volunteers to avoid missing areas in need.

FR10: The system shall allow different parts of the software to talk to each other to keep information up to date.

Team Member Roles

Project Members and Roles

Name	Role	description
ADLY AZAMIN BIN AZMAN	Project Manager	Define project scope and timeline, allocate responsibilities to team members, track progress and ensure the project is delivered on schedule.
MUHAMMAD UQAIL RAFIQIE BIN MOHD FARIS	Database Admin	Responsible for developing, managing, and maintaining the database system. Controls user permissions, protects data security and handles backup and recovery processes.
AFIQ FAHIM BIN MOHD RIDZUAN	System Architect	Determine how services are separated (e.g., user service, resource service), plan communication between services, select technologies and frameworks, ensure scalability and stability.